Software Requirements Specification

Log Analyzer & Report Generator (LARG)

Version: 1.1

---

1. Introduction

1.1 Scope

LARG is a desktop application designed to analyze log files and generate comprehensive reports. The system will:

- Import and parse log files in `.log` and `.txt` formats
- Support multiple parsing modes (template-based, generic, and custom regex)
- Enable users to search, filter, and sort log entries
- Generate analytics including entry counts, trends, and anomaly detection
- Export results in HTML and CSV formats

The application targets developers, system administrators, and QA engineers who need efficient tools for log file analysis and troubleshooting.

Out of Scope: Real-time log monitoring, cloud storage integration, multi-user collaboration, machine learning-based anomaly detection, and JSON/XML format support are not included in this version.

1.2 Definitions, Acronyms, and Abbreviations

- **GUI:** Graphical User Interface
- **CSV:** Comma-Separated Values
- **Log Entry:** A single line or record in a log file
- **Log Level:** Severity classification (e.g., INFO, WARNING, ERROR, CRITICAL)
- **Template Mode:** Parsing method for structured logs following a predefined format

- **Generic Mode:** Parsing method for unstructured logs
- **Regex Mode:** Parsing method using user-defined regular expressions

1.3 Overview

The remainder of this document is organized as follows:

- **Section 2** provides a general description of the software, including product perspective, functions, user characteristics, constraints, and dependencies.
- **Section 3** details specific functional and non-functional requirements organized by feature area.
- **Section 4** outlines the priority of system features
- **Section 5** describes the external interface requirements for the system, including hardware, external software, and communications

2. General Description

2.1 Product Perspective

LARG is a standalone desktop application developed as an independent, self-contained system. It does not interface with external systems or databases. The application operates entirely on the local file system, reading log files from user-specified locations and writing reports to user-selected destinations.

System Interfaces:

- Operating System: Windows, macOS, or Linux
- File System: Local file system access for reading log files and writing reports

User Interfaces:

- Graphical desktop interface built with PyQt6
- Interactive charts and tables for data visualization
- File dialogs for import/export operations

Hardware Interfaces:

- Standard desktop/laptop computer with sufficient memory to load log files
- Display capable of rendering GUI at minimum 900x600 resolution

Software Interfaces:

- Python 3.10+ runtime environment
- PyQt6 framework for GUI components
- matplotlib library for chart generation

2.2 Product Functions

A summary of the major functions LARG performs:

1. **File Import:** Load one or more log files (`.log` or `.txt` format)
2. **Log Parsing:** Parse log entries using template, generic, or custom regex modes
3. **Search and Filter:** Find specific entries using keywords, log levels, and date ranges
4. **Sorting:** Order entries by timestamp, level, message, or source file
5. **Analytics:** Calculate statistics, identify patterns, and flag anomalies
6. **Visualization:** Display data through tables and charts
7. **Export:** Generate reports in HTML or CSV format

2.3 User Characteristics

Primary Users:

- **Developers:** Technical users with programming experience who need to debug application behavior
- **System Administrators:** IT professionals monitoring system health and troubleshooting issues
- **QA Engineers:** Testing professionals verifying application behavior and identifying defects

Expected Skill Level:

- Familiarity with log file formats and log analysis concepts
- Basic understanding of regular expressions (for Regex Mode)
- Comfortable using desktop applications

User Frequency: Daily to weekly usage during development, testing, or troubleshooting activities.

2.4 General Constraints

- **Hardware Limitations:** Application performance depends on available system memory. Very large log files (>1GB) may require significant processing time
- **Interface Requirements:** Must provide a GUI (command-line interface not required)
- **Parallel Operations:** Single user desktop application; no concurrent user support needed
- **Programming Language:** Python 3.10 or higher required
- **Development Standards:** Code must be modular, testable, and follow Python PEP 8 style guidelines

2.5 Assumptions and Dependencies

Assumptions:

- Users have valid log files in `.log` or `.txt` format
- Log files are text-based and use UTF-8 or compatible encoding
- Users understand basic log analysis concepts
- The system has sufficient disk space for generated reports

Dependencies:

- Python 3.10+ interpreter must be installed
- PyQt6 library must be available
- matplotlib library must be available
- PyInstaller required for building executable

3. Specific Requirements

3.1 Functional Requirements

3.1.1 File Import

FR-1.1: The system shall support importing one or more log files simultaneously.

- **Input:** User selects a file via file dialog
- **Processing:** System validates file extensions

- **Output:** Files loaded into memory for parsing

FR-1.2: The system shall accept files with `.log` and `.txt` extensions.

- **Input:** File path with extension
- **Processing:** Extension validation check
- **Output:** Acceptance or rejection based on extension

FR-1.3: The system shall reject files with unsupported extensions.

- **Input:** File with non-supported extension
- **Processing:** Extension check fails
- **Output:** File is skipped; user notified if appropriate

3.1.2 Log Parsing

FR-2.1: The system shall support three distinct parsing modes:

FR-2.1.1: Template Mode

- **Description:** Parse structured logs following the format `timestamp level message`
- **Input:** Log file with structured entries
- **Processing:** Apply predefined regex pattern to extract fields
- **Output:** Parsed entries with timestamp, level, and message fields

FR-2.1.2: Generic Mode

- **Description:** Parse unstructured logs as plain text
- **Input:** Log file with unstructured entries
- **Processing:** Treat entire line as message field
- **Output:** Parsed entries with message field only

FR-2.1.3: Regex Mode

- **Description:** Parse logs using user-defined regular expressions with named capture groups
- **Input:** Log file and user-provided regex pattern
- **Processing:** Apply custom regex to extract fields
- **Output:** Parsed entries with fields matching named groups

FR-2.2: The system shall extract the following fields from log entries when available:

- Timestamp: datetime when log entry was created
- Log level: INFO, WARNING, ERROR, CRITICAL, DEBUG, etc.
- Message: descriptive text of the log entry
- Source file: name of the file containing the entry
- Line number: position within the source file

3.1.3 Search and Filter

FR-3.1: The system shall allow filtering by keyword.

- **Input:** User-provided keyword string
- **Processing:** Search message field for keyword match
- **Output:** Subset of entries containing the keyword

FR-3.2: The system shall allow exclusion of entries containing specific keywords.

- **Input:** User-provided exclusion keyword string
- **Processing:** Remove entries whose message contains the keyword
- **Output:** Subset of entries not containing the keyword

FR-3.3: The system shall allow filtering by log level.

- **Input:** User-selected log level (e.g., ERROR, WARNING)
- **Processing:** Match entries with specified level
- **Output:** Subset of entries matching the log level

FR-3.4: The system shall allow filtering by date range.

- **Input:** Start date and/or end date
- **Processing:** Compare entry timestamps to date range
- **Output:** Subset of entries within the specified date range

3.1.4 Sorting

FR-4.1: The system shall support sorting entries by the following criteria:

- Timestamp (chronological order)
- Log level (severity order)
- Message content (alphabetical order)
- Source file (alphabetical order)

FR-4.2: The system shall support both ascending and descending sort orders.

- **Input:** Sort key and direction

- **Processing:** Apply sorting algorithm
- **Output:** Reordered list of entries

3.1.5 Analytics

FR-5.1: The system shall calculate and display the following analytics:

FR-5.1.1: Total Entry Count

- **Description:** Count of all log entries currently loaded or filtered

FR-5.1.2: Log Level Distribution

- **Description:** Count of entries grouped by log level (INFO, WARNING, ERROR, etc.)

FR-5.1.3: Source File Distribution

- **Description:** Count of entries grouped by source file name

FR-5.1.4: Top Recurring Messages

- **Description:** List of most frequently occurring messages with occurrence counts
- **Processing:** Normalize messages by replacing numbers with placeholders

FR-5.1.5: Time-Based Distribution

- **Description:** Count of entries bucketed by time intervals (hourly)
- **Processing:** Extract hour from timestamp and aggregate counts

FR-5.1.6: Flagged Entries

- **Description:** Subset of entries meeting flagging criteria
- **Criteria:** ERROR or CRITICAL level, or message contains keywords like “timeout,” “failed,” “exception,” or “denied”

3.1.6 Export

FR-6.1: The system shall export filtered results to HTML format.

- **Input:** Current filtered dataset
- **Processing:** Generate HTML document with embedded CSS
- **Output:** HTML file written to user-specified location

FR-6.2: The system shall export filtered results to CSV format.

- **Input:** Current filtered dataset
- **Processing:** Convert entries to CSV rows with headers
- **Output:** CSV file written to user-specified location

FR-6.3: Exported reports shall include summary statistics and entry details.

- **Content:** Report metadata, summary statistics, top messages, flagged entries, and full entry list

3.2 Non-Functional Requirements

3.2.1 Performance Requirements

NFR-1.1: The system shall parse files containing up to 100,000 log entries within 10 seconds.

- **Condition:** File size up to approximately 10MB
- **Measurement:** Time from file selection to completion of parsing
- **Platform:** Standard desktop computer (8GB RAM, modern CPU)

NFR-1.2: Search and filter operations shall complete within 2 seconds for datasets up to 100,000 entries.

- **Condition:** Any combination of search and filter criteria
- **Measurement:** Time from user action to display of filtered results
- **Platform:** Standard desktop computer (8GB RAM, modern CPU)

3.2.2 Interface Requirements

NFR-2.1: The system shall provide a graphical user interface (GUI).

- **Description:** All functionality accessible through visual interface elements
- **Framework:** PyQt6-based desktop application
- **Minimum Resolution:** 900x600 pixels

NFR-2.2: The system shall provide clear error messages for invalid inputs or file formats.

- **Description:** Error messages shall identify the problem and suggest corrective action
- **Display:** Modal dialog boxes or status bar notifications
- **Examples:** “Unsupported file format,” “Invalid regular expression syntax”

NFR-2.3: The system shall display progress indicators for long-running operations.

- **Description:** Visual feedback during file loading, parsing, and export operations
- **Implementation:** Status bar messages indicating current operation

3.2.3 Reliability Requirements

NFR-3.1: The system shall handle malformed log entries gracefully without crashing.

- **Behavior:** Entries that cannot be parsed shall be marked with a PARSE_ERROR level
- **Recovery:** System continues processing remaining entries
- **User Notification:** Error count displayed in summary statistics

NFR-3.2: The system shall validate user-provided regular expressions before applying them.

- **Validation:** Test regex compilation before processing
- **Error Handling:** Display error message if regex is invalid
- **Recovery:** Allow user to correct the regex pattern

3.2.4 Portability Requirements

NFR-4.1: The system shall run on Windows, macOS, and Linux operating systems.

- **Windows:** Windows 10 or higher
- **macOS:** macOS 10.14 or higher
- **Linux:** Modern distributions with Python 3.10+ support

NFR-4.2: The system shall require Python 3.10 or higher.

- **Justification:** Uses modern Python features (structural pattern matching, enhanced type hints)
- **Verification:** Application checks Python version on startup

3.2.5 Maintainability Requirements

NFR-5.1: The system shall have a modular code structure separating concerns.

- **Modules:**
  - Parsing and searching logic (`parsing_searching.py`)
  - Analytics and reporting logic (`analytics_reporting.py`)
  - User interface layer (`ui_workflow.py`, `ui_widgets.py`, `ui_helpers.py`, `ui_constants.py`)
  - Application entry point (`main.py`)
- **Benefit:** Facilitates independent testing, modification, and enhancement

NFR-5.2: The system shall include unit tests for core parsing, searching, and analytics functions.

- **Coverage:** Minimum of 70% code coverage for non-UI modules
- **Framework:** pytest-based test suite
- **Test Files:** `test_parsing_searching.py`, `test_analytics_reporting.py`, `test_exporters.py`

3.2.6 Design Constraints

DC-1: The application shall be built using PyQt6 for the user interface.

- **Justification:** Cross-platform GUI framework with comprehensive widgets

DC-2: The application shall use matplotlib for analytics visualizations.

- **Justification:** Industry-standard plotting library with PyQt6 integration

DC-3: The application shall be a standalone desktop application.

- **Constraint:** No web-based or mobile versions required
- **Deployment:** Executable file generated via PyInstaller

4. System Features by Priority

4.1 Essential Features (Must Have)

- File import (FR-1.1, FR-1.2, FR-1.3)
- Template mode parsing (FR-2.1.1, FR-2.2)
- Search and filter (FR-3.1, FR-3.2, FR-3.3, FR-3.4)
- Basic analytics (FR-5.1.1, FR-5.1.2)
- CSV export (FR-6.2)

4.2 Important Features (Should Have)

- Generic mode parsing (FR-2.1.2)
- Sorting (FR-4.1, FR-4.2)
- Advanced analytics (FR-5.1.3, FR-5.1.4, FR-5.1.5, FR-5.1.6)
- HTML export (FR-6.1, FR-6.3)

4.3 Desirable Features (Nice to Have)

- Regex mode parsing (FR-2.1.3)
- Data visualizations
- Custom flagging keywords

5. External Interface Requirements

5.1 User Interface

The user interface shall consist of:

- **Home Screen:** File selection interface for initial load
- **Dashboard:** Main workspace with:
    - File management toolbar
    - Parsing mode selector
    - Search and filter controls
    - Sort controls
    - Results table (scrollable)
    - Summary statistics cards
    - Analytics charts
    - Export buttons
- **Status Bar:** Operation feedback and system messages

5.2 Hardware Interface

No special hardware interfaces required. Standard keyboard, mouse, and display.

5.3 Software Interface

Operating System:

- File system access for reading log files and writing reports
- Standard file dialogs for file selection

Python Libraries:

- PyQt6 (version 6.x): GUI framework
- matplotlib (version 3.x): Charting and visualization
- Python standard library: re, csv, datetime, pathlib, collections

5.4 Communications Interface

Not applicable. LARG operates as a standalone application with no network communication requirements.